AgentSelectionBench: Who Should Execute? Evaluating Capability–Objective Alignment

Anonymous ACL submission

Abstract

Language-model applications have expanded from generating responses to executing tasks through tools and agent harnesses. This broadens selection from choosing a model to choosing an execution configuration whose suitability depends on the required outcome and execution conditions. We introduce AgentSelectionBench, a benchmark for agent selection under heterogeneous objectives, connecting these specifications to verifiable execution evidence. Recorded native acceptance results and resource measurements determine which candidates qualify and how economically they do so. The benchmark organizes 1,116 source tasks from nine task families into 1,180 packages and 22,240 constrained scenarios, with historical evidence separated from target outcomes. It evaluates objective satisfaction and cost efficiency, and uses relations between qualifying candidate sets to assess adaptation across conditions. Across eight recorded selector systems, satisfaction and efficiency yield different rankings. Condition-specific choices improve satisfaction by 0.71–14.60 percentage points over reusing unconstrained choices, yet the best selector fully covers only nine of 114 target groups requiring candidate changes across feasible conditions. These results identify a gap between benefiting from execution conditions in aggregate and selecting appropriately throughout their variation, extending agent selection evaluation beyond an overall performance ranking.

1 Introduction

Routing has provided a practical way to exploit the growing diversity of language models: assign a query to a suitable model and balance response quality against the cost of inference (Hu et al., 2024; Ong et al., 2025). As language-model applications extend into tool use and interaction with external environments, the system receiving a task increasingly includes an execution harness that coordinates model calls, tools, and intermediate state. This development broadens the selection problem from comparing model backends to comparing executable agent configurations (Kapoor et al., 2026; Shi et al., 2026).

The expansion changes the evidence needed to evaluate a choice. A harness determines how an agent manages context, accesses tools, and responds to execution feedback; different harnesses can therefore produce different outcomes and resource use with the same backbone model (Yao et al., 2026). Task completion may depend on a tested code change, a computed artifact, or a verified environment state. Assessing a configuration consequently requires evidence about what its execution achieved and what that achievement consumed. Model-level response scores remain useful, but do not by themselves characterize these execution outcomes. Cross-benchmark execution records and controlled harness evaluations now provide a basis for studying this richer decision (Kapoor et al., 2026; Yao et al., 2026).

Execution evidence, in turn, must be interpreted relative to the objective being selected for. A scientific task can require a complete solution or an independently checked component; the latter has its own acceptance scope (Tian et al., 2024). Even for a fixed outcome, different cost and time limits can change which executions qualify. Among qualifying alternatives, expenditure can still differ substantially. These distinctions make an overall configuration ranking insufficient to determine the appropriate choice. They also make variation in selection ambiguous: keeping one configuration may be correct across several conditions, whereas another set of conditions may require switching to preserve satisfaction.

Recent work already studies query-conditioned agent recommendation and joint model–harness routing (Shi et al., 2026; Chen et al., 2026). Build-

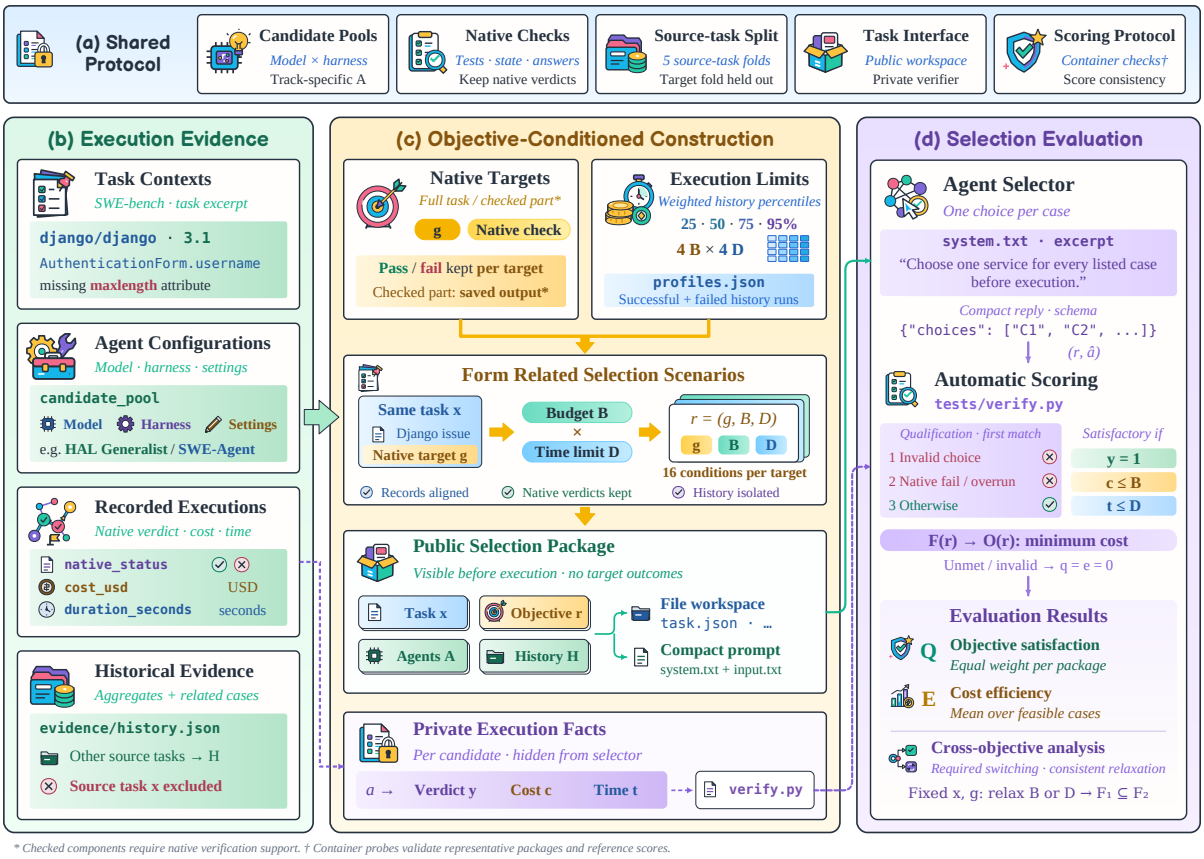


Figure 1: **Overview of AgentSelectionBench.** (a) Shared protocols specify candidate pools, native checks, source-task separation, and evaluation. (b) Task contexts, agent configurations, and recorded executions provide acceptance and resource evidence; other-task history supplies information for selection. (c) Native targets and history-calibrated execution conditions form related selection scenarios, with public inputs separated from private candidate facts. (d) Submitted choices are evaluated for objective satisfaction Q and cost efficiency E. Q averages within packages and then across packages; E pools feasible scenarios. The same reference sets support analyses of necessary switching and consistency under relaxed constraints.

ing on this progress, we ask: *can a selector identify a satisfactory and economical agent for a specified objective, and adjust appropriately when its execution conditions change?* We introduce AgentSelectionBench for *agent selection under heterogeneous objectives*. It treats the required outcome and execution constraints as parts of the selection instance, so that correctness follows from whether the chosen configuration fulfills that specification. The benchmark evaluates both individual choices and related choices sharing a task context.

We construct the benchmark from agent executions in native task environments. Acceptance verdicts establish which outcomes were achieved, and associated cost and duration records establish whether they meet the specified conditions. Historical executions calibrate the conditions and provide selection evidence, with source-task separation keeping target outcomes private. The collection covers nine task families, comprising 1,116 source tasks, 1,180 task packages, and 22,240 constrained scenarios. Deterministic scoring measures objective satisfaction and cost efficiency. The qualifying candidate sets additionally identify when a common choice remains appropriate and when complete coverage requires changing candidates. Under constraint relaxation, a satisfactory tighter-condition choice remains a feasible alternative, providing a direct witness for avoidable regressions.

Evaluation of eight recorded selector systems reveals that satisfaction and cost efficiency produce different rankings. Condition-specific choices improve satisfaction by 0.71–14.60 percentage points over reusing each selector's unconstrained choice for the same target. These gains nevertheless leave substantial adaptation failures: among 114 target groups requiring candidate changes to cover every feasible condition, the best selector fully cov-

ers only nine. Recorded choices can also lose satisfaction under relaxed constraints, reaching 9.63% of comparisons where the tighter-condition choice qualifies. These results distinguish improvements in aggregate selection from correct adaptation across execution conditions.

2 Related Work

2.1 Model routing and resource-aware selection

RouterBench and LLMRouterBench standardize quality–cost evaluation across model choices, while RouterEval examines gains from model complementarity (Hu et al., 2024; Li et al., 2026; Huang et al., 2025). RouteLLM learns routing from preference data; VL-RouterBench extends evaluation to vision-language models and router throughput (Ong et al., 2025; Huang et al., 2026). RADAR and SCORE further incorporate reasoning-budget allocation and performance, cost, or latency constraints (Fernandez et al., 2026; Lakha et al., 2025). These studies establish resource-aware routing as an existing problem. Their model and inference-level comparisons provide useful foundations, while agent selection additionally requires connecting resources to outcomes produced by a model–harness execution. We evaluate this connection through native acceptance targets and the expenditure associated with the selected outcome.

2.2 Agent recommendation and execution configurations

AgentSelect represents agents through model–tool capability profiles, converts heterogeneous evaluation artifacts into recommendation supervision, and validates recommendations in marketplace and deployment settings (Shi et al., 2026). HM-Router learns task-dependent model–harness compatibility, including cost-aware selection, unseen combinations, and new components (Chen et al., 2026). Execution studies supply complementary evidence: HAL standardizes cross-benchmark trajectories and outcome measurements, while Harness-Bench evaluates completion requirements, configuration quality–cost frontiers, and model-specific harness selection on held-out tasks (Kapoor et al., 2026; Yao et al., 2026). These works establish both configuration selection and its dependence on execution behavior. Our benchmark extends the evaluation unit from a task–configuration observation to a family of explicitly specified selection instances, allowing the same recorded capabilities to be assessed against different acceptance targets and execution conditions.

2.3 Outcome granularity and behavioral evaluation

SciCode provides native checks at problem and subproblem granularity, and ProCredit applies acceptance checks to intermediate states for progress measurement and training credit (Tian et al., 2024; Ma et al., 2026). We preserve the acceptance scope of these outcomes when constructing references for agent selection. Behavioral evaluation addresses a different limitation of aggregate scores: CheckList tests invariance and directional expectations, while RouteJudge diagnoses selection collapse, distributions, and inter-router agreement (Ribeiro et al., 2020; Lai et al., 2026). Our evaluation connects these perspectives through objective-specific reference sets. Intersections of the reference sets distinguish justified persistence from necessary switching. Nesting of the satisfactory sets under constraint relaxation identifies regressions with a verified feasible alternative. This supplies correctness criteria for selection behavior that choice variation alone cannot provide.

3 Task Formulation: Agent Selection under Heterogeneous Objectives

Let $x \in \mathcal{X}$ denote a task context and $\mathcal{G}(x)$ the objectives supported by that context. A selection instance specifies $r = (g, B, D)$, where $g \in \mathcal{G}(x)$ identifies the required outcome and acceptance scope, and B and D constrain execution cost and time. Let $\mathcal{A}_b$ be the candidate-agent set for benchmark–track pool b. Each candidate is a model–harness configuration associated with its recorded tools and execution settings. Given the context, specification, candidate descriptions, and permitted historical execution evidence $\mathcal{H}_{-x}$ excluding the current source task, the selector returns

$$\hat{a} = \pi(x, r, \mathcal{A}_b, \mathcal{H}_{-x}). \tag{1}$$

The formulation in Figure 1(c) separates the underlying task context from the current objective: one context can support objectives with different functions or scopes, and one objective can be subject to different execution conditions. The selector uses configuration information and historical behavior to determine which candidate capabilities fit the current specification. We evaluate a collection of

selectors Π on a common set of instances; the selected configuration $a \in \mathcal{A}_b$ is the execution agent, while $\pi \in \Pi$ makes the selection.

We define candidate suitability through objective-conditioned execution feedback. Write $z_{xga} = (y_{xga}, c_{xga}, t_{xga})$, where y_{xga} is the native acceptance verdict for the objective, and c_{xga} and t_{xga} are the associated execution cost and duration. For scenario $i = (x, g, B_i, D_i)$, abbreviate these quantities as y_{ia}, c_{ia}, and t_{ia}. Its satisfactory set is

$$\mathcal{F}_i = \{a \in \mathcal{A}_b : y_{ia} = 1,\ c_{ia} \le B_i,\ t_{ia} \le D_i\}\,. \tag{2}$$

The primary objective is to select $\hat{a}_i \in \mathcal{F}_i$; when several candidates qualify, we additionally compare their cost efficiency. The set admits multiple valid choices, while an empty set indicates that the observed candidate executions do not satisfy the specification. The benchmark establishes these relations from execution evidence and evaluates whether selectors can identify qualifying, economical candidates without observing the target outcomes.

4 Benchmark Construction

4.1 Execution evidence and task environments

Our data are grounded in actual agent executions in the source benchmarks' task environments, covering software development, scientific computing, information seeking, web interaction, and service operations. These environments give tasks their operational meaning: coding tasks depend on repositories and tests, scientific tasks on data and computational resources, and interactive tasks on tool interfaces and environment state. Candidate agents execute under these conditions, and the corresponding benchmarks' native acceptance mechanisms assess their outcomes. We assemble task, configuration, and execution records from existing evaluation runs, linking outcome verdicts to observed costs and durations as the evidence base for selection instances (Kapoor et al., 2026; agent-evals, 2026).

To connect heterogeneous sources, we index execution evidence by (x, g, a), aligning each task context, acceptance target, and candidate identity with its feedback z_{xga}. The model and harness (m, h) form the configuration core: the model supplies reasoning and knowledge capabilities, while the harness organizes tool access, context management, and execution. Associated runtime settings remain configuration metadata. Environment information is organized by its role in the task, including problem context, available resources, operational interfaces, and acceptance targets. This preserves task-specific semantics and native checks while providing a common representation of the relation between objectives, candidates, and execution outcomes.

4.2 Objective-conditioned construction and validation

The construction links the recorded execution evidence in Figure 1(b) to the objective-conditioned selection instances in Figure 1(c). We represent the dataset as

$$\begin{aligned} \mathcal{D} &= \{(x_i, r_i, \mathcal{A}_{b_i}, \mathcal{H}_{-x_i}; \mathcal{Z}_i)\}_{i=1}^{N_s}\,, \\ \mathcal{Z}_i &= \{(a, z_{x_i g_i a}) : a \in \mathcal{A}_{b_i}\}\,, \end{aligned} \tag{3}$$

where N_s is the number of selection scenarios. The fields before the semicolon are visible to the selector; $\mathcal{Z}_i$ contains the candidate execution facts retained for evaluation. Native feedback supplies supervision on whether the required outcome was achieved, while cost and duration establish whether that outcome qualifies under the specified conditions. Together they induce the satisfactory set in Eq. 2 and support efficiency comparisons among qualifying candidates.

Construction first determines $\mathcal{G}(x)$ from the original task description and acceptance structure, organizing complete outcomes and natively supported finer-grained outcomes into explicit objectives. Each objective is linked to the corresponding candidate feedback. We then calibrate cost and time conditions from permitted historical executions and combine them with objectives to form selection scenarios. Each resource uses the weighted empirical 25th, 50th, 75th, and 95th percentiles, including successful and failed runs. Source tasks receive equal total weight; candidates divide their task's weight, and saved states divide their candidate's weight. Known costs and durations are normalized separately when computing their quantiles. The construction thus represents required function, outcome scope, and execution constraints as distinct factors: changing g changes the required outcome, whereas changing (B, D) changes its execution conditions. The same execution evidence can support judgments under multiple specifications, with each judgment grounded in a defined acceptance target and associated behavior. Resource

conventions and evidence scope are illustrated in Appendix B.

Historical evidence and evaluation instances are separated by source task, with all objectives and condition combinations derived from a source task assigned together. Records outside the target partition support historical statistics, related-case retrieval, and resource calibration. We organize this information into selection task packages: public materials provide the task context, objective specification, candidate descriptions, and historical evidence, while private materials retain target execution facts and evaluation logic. Historical aggregates summarize domain-level behavior; related cases provide task-level execution experience. The selector submits a candidate identifier, and the evaluator scores it against the corresponding specification. This connects evidence produced in native task environments to a reusable interface for selection evaluation.

Environment handling and validation center on this interface. Task packages separate the public workspace from private evaluation materials and provide a common submission protocol and automatic scorer, with support for isolated container execution and verification. During construction, automated checks validate task–candidate–objective associations, preservation of native verdicts, and separation of historical evidence. Independently computed expectations check scoring consistency. Representative packages additionally undergo end-to-end container tests covering material access, reference submissions, and score generation. Interface details and settings are described in Appendix C.

4.3 Benchmark characteristics

Appendix Figure 6 summarizes the collection and its selection structure. It spans AssistantBench, CORE-Bench, GAIA, Online-Mind2Web, SciCode, ScienceAgentBench, SWE-bench Verified Mini, τ-bench Airline, and USACO (Yoran et al., 2024; Siegel et al., 2024; Mialon et al., 2023; Xue et al., 2025; Tian et al., 2024; Chen et al., 2024; Jimenez et al., 2024; Yao et al., 2025; Shi et al., 2024), comprising 1,116 source tasks, 1,180 packages, and ten benchmark–track pairs. SciCode's full-task and saved-output tracks share 64 source tasks and support whole-task and finer-grained objectives, respectively. Each of the 1,390 main task–target groups is paired with four budget and four duration levels, producing 22,240 constrained scenarios; 1,454 unconstrained quality-baseline choices are retained separately. Source-task, objective-group, and scenario counts describe the underlying work, acceptance structure, and execution-condition coverage, respectively. Appendix Table 3 provides the detailed counts.

The benchmark exposes both objective feasibility and the choice space among qualifying candidates. For the same task–target groups, feasible-scenario coverage is 36.19% under the tightest cost–time combination and 67.63% under the most permissive combination, showing how execution conditions change the usable range of candidate capabilities. In 8,452 scenarios, multiple candidates satisfy the same specification but have different execution costs. The dataset therefore supports evaluation of both objective satisfaction and selection efficiency: identifying suitable agents, then using resources efficiently within the qualifying set. Section 5 defines these metrics, and Section 6 analyzes how recorded selectors perform on these relations.

5 Evaluation Protocol

5.1 Objective satisfaction and cost efficiency

Evaluation measures whether a choice realizes the specified objective under its execution constraints and how efficiently it does so. As shown in Figure 1(d), scores are computed from the submitted service identity and private execution facts. A choice satisfies scenario i when it belongs to the satisfactory set $\mathcal{F}_i$ defined in Section 3:

$$q_i = \mathbf{1}[\hat{a}_i \in \mathcal{F}_i]. \tag{4}$$

The primary satisfaction score averages within each package and then gives equal weight to the $N = 1{,}180$ packages:

$$Q = \frac{1}{N} \sum_{p=1}^{N} \frac{1}{|\mathcal{I}_p|} \sum_{i \in \mathcal{I}_p} q_i, \tag{5}$$

where $\mathcal{I}_p$ contains package p's main constrained scenarios. This prevents packages with more sub-targets from dominating solely through scenario count. Invalid or missing submissions receive zero; unconstrained baselines do not enter Q.

Economic efficiency compares choices after satisfaction has been established. For each nonempty $\mathcal{F}_i$, define the minimum satisfactory cost and the services attaining it:

$$\begin{aligned} c_i^\star &= \min_{a \in \mathcal{F}_i} c_{ia}, \\ \mathcal{O}_i &= \{a \in \mathcal{F}_i : c_{ia} = c_i^\star\}. \end{aligned} \tag{6}$$

Let $\mathcal{I}_F = \{i : \mathcal{F}_i \neq \varnothing\}$ be the feasible scenarios. The efficiency score is

$$e_i = \begin{cases} c_i^\star / c_{i\hat{a}_i}, & \hat{a}_i \in \mathcal{F}_i, \\ 0, & \text{otherwise}, \end{cases} \qquad E = \frac{1}{|\mathcal{I}_F|} \sum_{i \in \mathcal{I}_F} e_i. \tag{7}$$

A zero-cost to zero-cost ratio is defined as one. A cheap choice that fails to satisfy the scenario receives no efficiency credit. E pools feasible scenarios and combines satisfaction with relative expenditure; it is not a percentage of money saved. We report Q and E separately and use Q for the primary ordering.

To interpret E, let H be the fraction of feasible scenarios on which the selected service is satisfactory, and C the mean ratio $c_i^\star / c_{i\hat{a}_i}$ over those satisfactory selections, taking $C = 0$ if there are none. Then $E = H\,C$. This distinguishes finding a satisfactory service from choosing an economical one. Because Q uses package-macro aggregation while H pools feasible scenarios, Q cannot replace H in this identity.

5.2 Reference sets and attainable coverage

Ground truth consists of candidate execution facts and the sets induced by each scenario. Every selected member of $\mathcal{F}_i$ earns satisfaction credit, while $\mathcal{O}_i$ identifies the minimum-cost choices and allows ties. The protocol therefore does not require a unique correct service. These target facts and reference sets remain private to the evaluator and are shared across all $\pi \in \Pi$. Comparing additional selectors therefore preserves the objective specifications and reference choices.

The observed-pool oracle assigns

$$q_i^{\text{oracle}} = \mathbf{1}[\mathcal{F}_i \neq \varnothing] \tag{8}$$

and uses the same package-macro aggregation as Q. It measures satisfaction coverage attainable from the recorded candidate pool. Empty-set scenarios remain in Q and its oracle, but are excluded from H and E. This separates limitations of the offered services from missed opportunities to select a satisfactory service.

6 Experiments and Results

We evaluate agent selection along the two axes established in Sections 3–5: realizing a specified objective under execution constraints, and choosing economically among satisfactory candidates. We first compare overall and task-family performance, then examine how choices respond to changing conditions and where they fail.

6.1 Experimental setup

Selectors and coverage. We evaluate eight LLM-based selectors: GPT-6.1-sol, Claude-Sonnet-5-5, Gemini-3.8-flash, Qwen3-VL-Plus, GLM-5.3, DeepSeek-v4.1-Flash, Kimi-K3.1-Pro, and Doubao-Seed-2.1-Pro. Each selector is evaluated on the same 1,180 task packages across nine benchmark families, yielding 9,440 selector–package evaluations and 177,920 scenario–selector evaluations. The 22,240 scenarios per selector share the candidate pools, objective specifications, and private execution facts defined in Section 4.

Selection protocol. Each package supplies task context, acceptance targets, candidate configurations, cost–time conditions, and historical evidence from other source-task folds. The selector returns one candidate identifier for each scenario, including unconstrained reference scenarios. The recorded suite uses file-access and compact direct-input interfaces; the latter presents historical aggregates and up to two retrieved cases. All 45 CORE packages use the same revised candidate pool and compact interface. Exact input and submission settings appear in Appendix C. Results characterize these instantiated selector systems with their recorded evidence-access settings.

Metrics. We report package-macro objective satisfaction Q and feasible-scenario cost efficiency E, together with the decomposition $E = HC$ from Section 5. H measures satisfaction on feasible scenarios, and C measures cost efficiency conditional on a satisfactory choice. The decomposition uses 0–1 values; all four metrics are displayed on a 0–100 scale. Resource quantities refer to the candidate execution associated with the selected outcome. Missing or invalid submissions remain in the evaluation and receive zero. Additional diagnostics reuse the same candidate facts and submitted choices.

6.2 Overall and task-family performance

Satisfaction and efficiency induce different rankings. Table 1 reports the eight selectors under the common evaluation protocol. GPT and Gemini attain the highest Q, at 30.71 and 30.53, whereas GLM leads E at 33.63. The decomposition explains the difference: GPT satisfies 55.79% of fea-

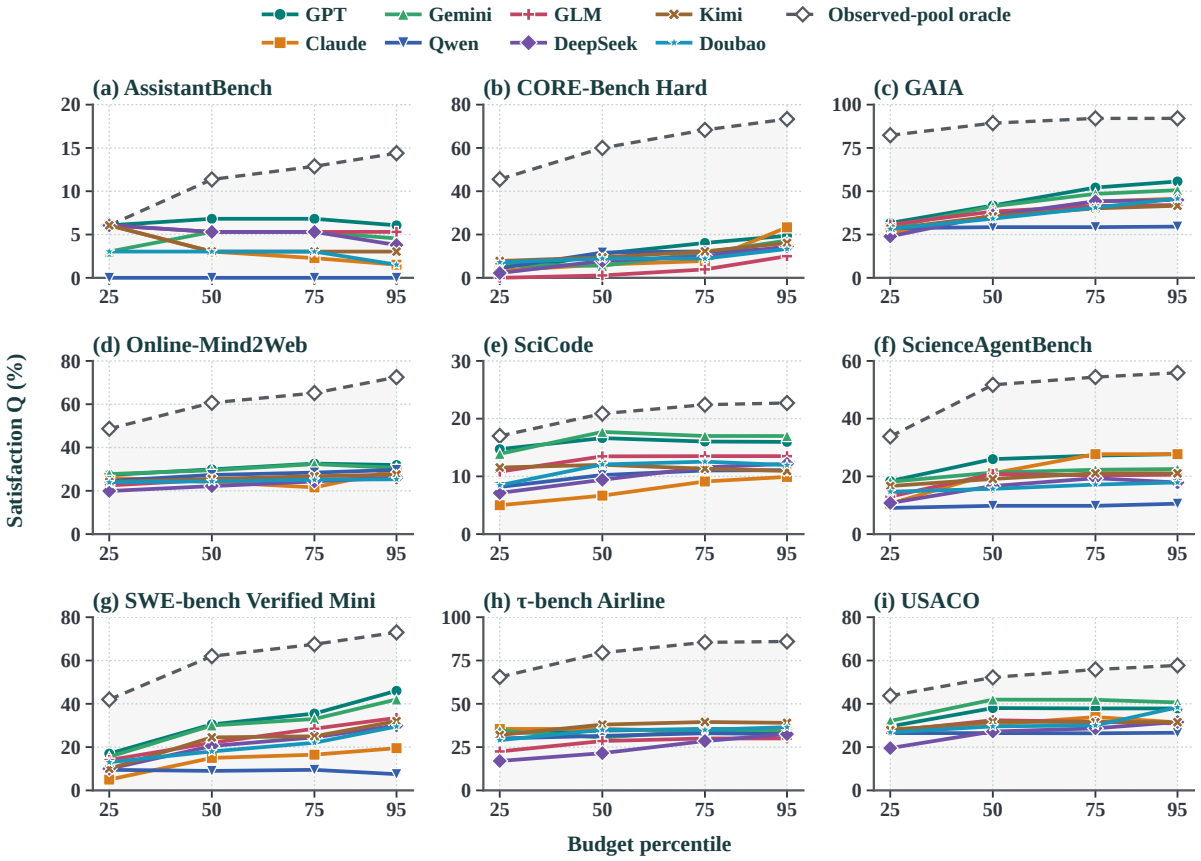


Figure 2: **Budget-conditioned objective satisfaction across nine task families.** At each historical budget percentile, we average the four deadline levels and all main targets within each package, then average packages equally within the family. Dashed curves and light shading show observed-pool oracle coverage. Panels use different vertical ranges; SciCode combines both tracks. Lines connect the four recorded conditions without smoothing.

sible scenarios, compared with GLM's 47.19%, but its conditional efficiency is 54.38%, compared with GLM's 71.26%. Qwen has the highest conditional efficiency ($C = 76.92$), while satisfying only 41.29% of feasible scenarios. The same distinction appears among selectors with similar satisfaction: Claude, GLM, and Doubao obtain Q between 24.95 and 25.06, yet their E scores range from 23.15 to 33.63. Objective satisfaction alone therefore leaves substantial differences in the economic value of selections unresolved. The observed-pool oracle has $Q = 56.80$, leaving a 26.09-point gap above the best recorded selector under the same aggregation.

Task families expose different selection difficulties. Figure 3 shows that the leading selector depends on both the task family and the evaluation criterion. On GAIA, GPT leads satisfaction ($Q = 45.31$), while Qwen leads efficiency ($E = 27.11$). Kimi leads satisfaction on τ-bench Airline ($Q = 37.13$) and efficiency on ScienceAgentBench, SWE-bench Verified Mini, and USACO ($E = 24.53$, 15.18, and 46.33); Doubao leads efficiency on Online-Mind2Web ($E = 28.02$). These differences are obscured by a single aggregate ordering.

Selector	Q	E	H	C
GPT	**30.71**	30.34	55.79	54.38
Claude	25.03	23.15	43.23	53.55
Gemini	30.53	28.35	**57.00**	49.74
Qwen	22.01	31.76	41.29	**76.92**
GLM	25.06	**33.63**	47.19	71.26
DeepSeek	22.95	21.32	40.75	52.33
Kimi	25.65	29.51	44.69	66.03
Doubao	24.95	29.84	44.94	66.39

Table 1: Overall selection results (0–100). Q is package-macro objective satisfaction; H is satisfaction on feasible scenarios, and C is conditional cost efficiency. On the 0–1 scale, $E = HC$. All selectors use the same 1,180 packages and 11,589 feasible scenarios. Bold marks column maxima; exact model identifiers appear in the setup.

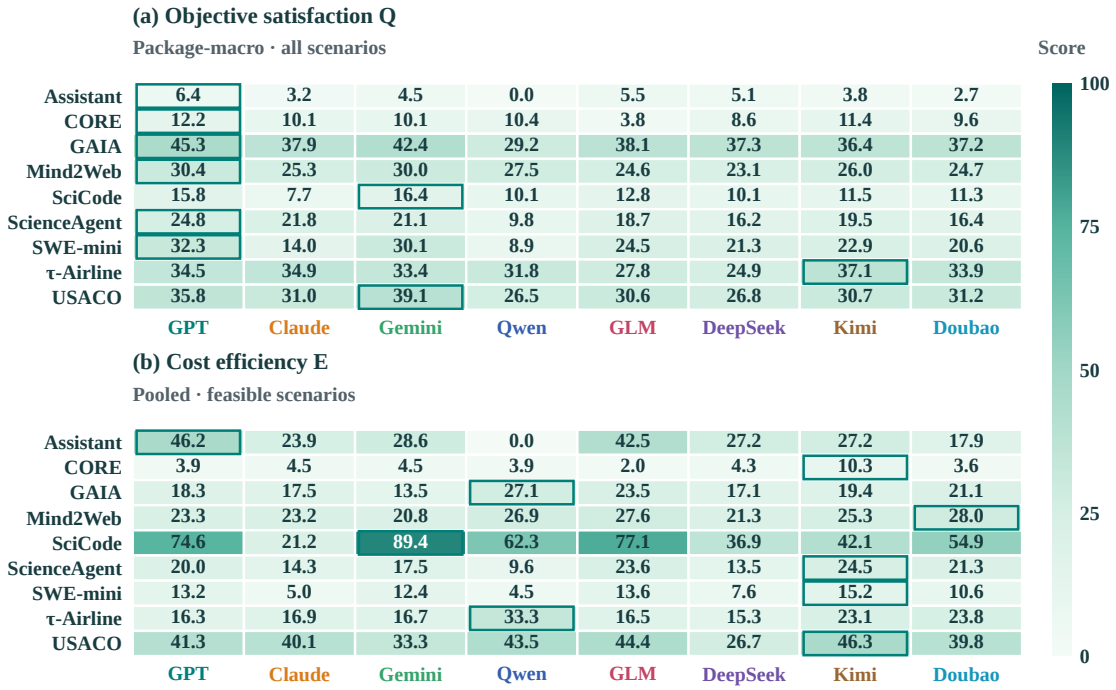


Figure 3: **Selection performance across task families.** (a) Objective satisfaction Q, averaged over packages within each family. (b) Cost efficiency E, pooled over feasible scenarios within each family. Values use a 0–100 scale; outlined cells mark the best selector in each task-family row. The two panels retain their distinct denominators. SciCode combines its full-task and saved-output tracks; ScienceAgent denotes ScienceAgentBench and SWE-mini denotes SWE-bench Verified Mini.

CORE has low satisfaction across selectors, with a best Q of 12.22, but its efficiency scores are heterogeneous: Kimi reaches $E = 10.30$, while the others remain below 4.50. SciCode instead combines limited attainable coverage (oracle $Q = 20.75$) with high efficiency on feasible scenarios, reaching $E = 89.35$ for Gemini. Coverage of objectives and economy on attainable objectives are thus distinct properties of a selector–task-family pairing.

6.3 Selection under changing execution conditions

Figure 2 shows how attainable coverage and realized selection vary with budget while retaining the same task groups. Larger allowances expand the satisfactory candidate sets, but recorded choices need not improve monotonically. For example, SciCode's GPT score declines from 16.63 at the 50th budget percentile to 15.97 at the 95th, although oracle coverage rises from 20.87 to 22.71. The within-target consistency analysis below examines this behavior directly.

Condition-specific choices improve over reusing an unconstrained choice. For each task–target group, we take the selector's recorded choice for the corresponding unconstrained scenario and reuse it across all sixteen constrained scenarios. We score this repeated choice and the actual condition-specific choices against the same execution facts and average both using Q's package weights. Every main target has a matching unconstrained reference with identical candidate identities and outcome facts. Both decisions come from the same recorded submission, so the comparison measures their realized difference rather than the causal effect of changing the input.

Figure 4(a) shows positive realized gains for all eight selectors, ranging from 0.71 points for Qwen to 14.60 for Gemini. GPT gains 14.35 points, Kimi 10.12, GLM 9.57, Doubao 9.37, Claude 6.42, and DeepSeek 3.04. For example, GPT rises from 16.36 under the repeated choice to 30.71 under its scenario-specific choices. Although the size of the gain varies, condition-specific choices outperform unconstrained-choice replay in aggregate for every evaluated system. This comparison does not require the best choice to change in every scenario: its benefit can also come from finding a suitable candidate shared by several conditions.

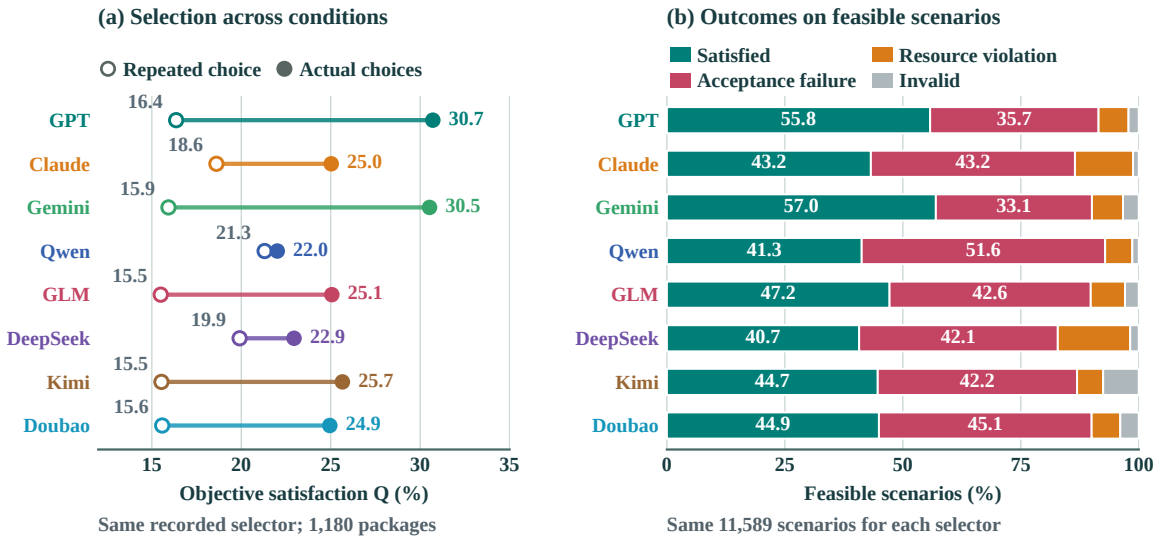


Figure 4: **Condition-specific selection and remaining errors.** (a) Package-macro Q of recorded scenario-specific choices versus replaying the same selector's unconstrained choice for each task–target group. Both use identical constrained scenarios; this is a recorded-choice comparison. (b) Mutually exclusive outcomes on the same 11,589 feasible scenarios per selector. The satisfactory share is H, not Q; resource violations occur after native acceptance, and invalid includes absent submissions.

6.4 Adaptation and consistency across conditions

Changing a choice is not the same as adapting correctly. The satisfactory and minimum-cost sets let us identify when adaptation is actually necessary. Among 940 task–target groups with at least one feasible condition, 715 share a minimum-cost satisfactory candidate across all feasible conditions. Another 111 admit a common satisfactory candidate but no common minimum-cost choice; the remaining 114 have no common satisfactory candidate. The last group necessarily requires changing candidates to cover all feasible conditions. Table 2 shows that GPT changes its choice in 97 of 114 valid groups and Gemini in 102 of 110, yet they satisfy every feasible condition in only nine and eight of the 114 groups, respectively. Kimi and DeepSeek completely satisfy three and one groups, respectively, while Claude, Qwen, GLM, and Doubao satisfy none. Doubao changes candidates in 101 of 113 valid groups, illustrating the gap between frequent switching and complete coverage. Selection variation therefore needs to be evaluated against the target-specific reference sets, rather than treated as successful adaptation by itself.

Looser conditions can expose avoidable regressions. For a fixed context and target, increasing either the budget or deadline while holding the other fixed gives $\mathcal{F}_i \subseteq \mathcal{F}_j$. A satisfactory choice at the tighter condition remains a valid witness at the looser one. We evaluate the 33,360 adjacent strict relaxations $\mathcal{L}$ across the task–target grids and define

$$R_{\downarrow} = \frac{\sum_{(i,j)\in\mathcal{L}} q_i(1-q_j)}{\sum_{(i,j)\in\mathcal{L}} q_i}. \qquad (9)$$

This pools comparisons within recorded submissions, with tighter-condition successes as the denominator (undefined if none exist). Comparisons share source executions; they are not additional independent trials. GPT regresses on 805 of 9,297

Selector	Switch/valid	Complete	$R_{\downarrow}$ (%)
GPT	97/114	9	8.66
Claude	94/114	0	8.05
Gemini	102/110	8	9.63
Qwen	17/114	0	1.72
GLM	47/113	0	3.78
DeepSeek	29/113	1	4.10
Kimi	65/110	3	6.75
Doubao	101/113	0	9.38

Table 2: Adaptation and consistency diagnostics. Switch/valid counts groups where choices differ across feasible conditions, among valid submissions to the 114 groups requiring a switch for full coverage. Complete counts groups satisfying every feasible condition, out of all 114. $R_{\downarrow}$ is the avoidable-regression rate across adjacent relaxed conditions over the full suite (Eq. 9); its denominator is the selector's tighter-condition successes.

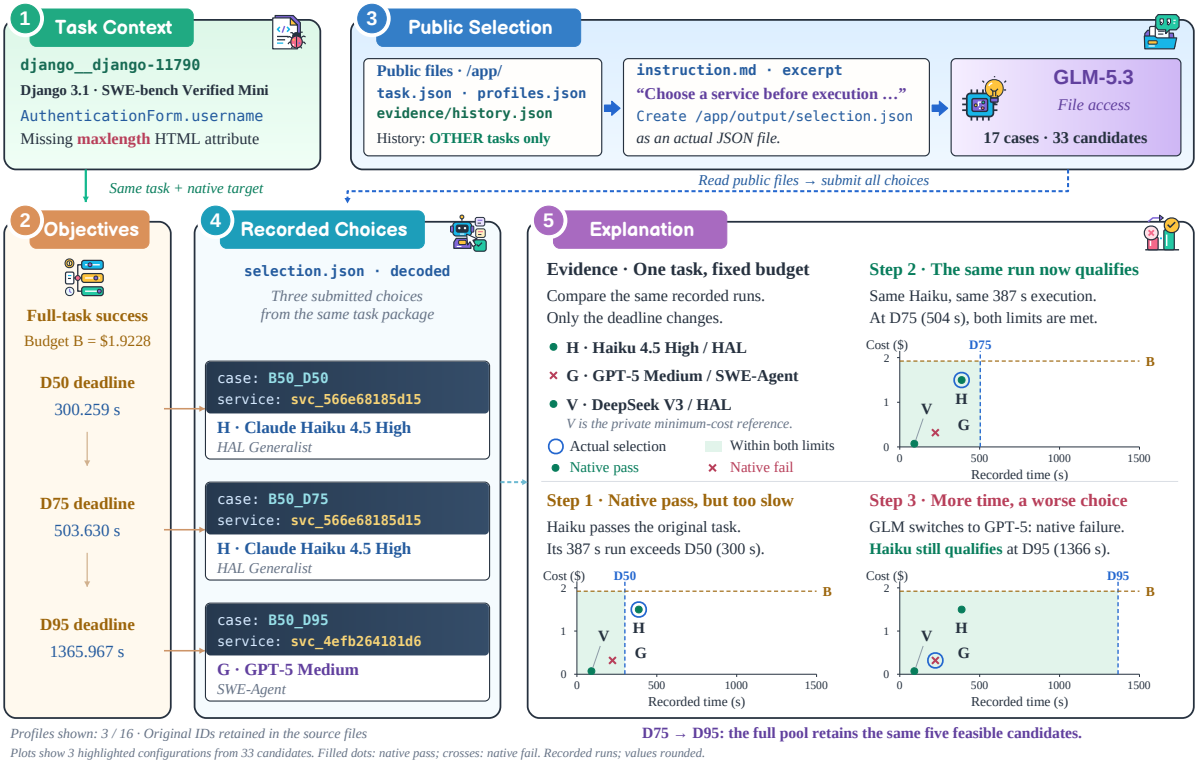


Figure 5: **A relaxed deadline can expose an avoidable selection regression.** Three recorded GLM-5.3 choices for Django task `django__django-11790` share the full-task target and a budget of $1.923. The plots highlight three of 33 configurations on identical axes. Green circles and red crosses denote native acceptance and failure; blue rings mark recorded selections. Shading indicates resource compliance. The choices yield $(q_i, e_i) = (0, 0)$, $(1, 0.048)$, and $(0, 0)$ as the deadline increases. The latter two profiles have the same five satisfactory candidates, including the previously selected H. V is the private minimum-cost reference, not a selector choice. Displayed values are rounded; scoring uses exact records.

eligible comparisons (8.66%), and Gemini on 918 of 9,534 (9.63%). Across all eight selectors, 4,189 such regressions include 3,161 switches to a native-failing candidate and 1,028 to a resource-infeasible candidate. Each has a verified satisfactory alternative supplied by the selector's own tighter-condition choice. Qwen's lower regression rate (1.72%) accompanies its small gain from condition-specific selection (0.71 points); stability alone therefore cannot establish effective adaptation. These diagnostics jointly assess whether a selector changes when needed and preserves already attainable satisfaction when conditions become less restrictive.

6.5 Error analysis

Most missed opportunities involve the wrong outcome, not only excess expenditure. Among feasible scenarios where a selector fails, choosing a candidate that fails the native target accounts for 71.05%–87.86% of errors across the eight systems. Figure 4(b) separates these cases from native-passing choices that violate execution constraints and from invalid submissions. For GPT, the 11,589 feasible scenarios comprise 6,465 satisfactory choices, 4,137 native-target failures, 736 resource violations, and 251 invalid choices. Native acceptance failure is the largest error category for every selector. The remaining composition differs: DeepSeek incurs 1,778 resource violations, while Kimi has 874 invalid choices, both counted on the same feasible-scenario denominator. Separating these outcomes distinguishes candidate-outcome matching, constraint compliance, and submission reliability within the common selection evaluation.

Case study: qualification, efficiency, and avoidable regression. Figure 5 traces three choices from one GLM-5.3 submission for a Django authentication-form repair. The native full-task target and budget remain fixed (approximately $1.923). At a deadline of 300.259 seconds, GLM selects H, a Claude Haiku 4.5 High/HAL configuration that passes native acceptance but requires 387.162 seconds, yielding $(q_i, e_i) = (0, 0)$.

With the deadline relaxed to 503.630 seconds, the same choice qualifies. Its cost is approximately $1.498, compared with $0.072 for the minimum-cost satisfactory candidate, giving $e_i \approx 0.048$. At 1365.967 seconds, GLM instead selects G, a GPT-5 Medium/SWE-Agent configuration that meets both resource limits but fails the native target. The latter two conditions retain exactly the same five satisfactory candidates, and H remains valid. The resulting loss of satisfaction therefore has a verified alternative supplied by the selector's own earlier choice. These profiles distinguish native acceptance, constraint compliance, and economic efficiency using the same recorded executions. Appendix Table 8 provides a complementary SciCode example: independently checked subtargets expose capability differences hidden by identical full-task failures, and choices must be assessed against each target's own reference set.

7 Discussion: Objective-Conditioned Agent Selection

The eight-selector evaluation exposes three complementary properties of agent selection: identifying a candidate that realizes the required outcome, choosing economically within the satisfactory set, and maintaining appropriate choices as execution conditions change. Aggregate satisfaction, conditional economy, and cross-condition consistency capture different aspects of this problem. Their separation makes it possible to distinguish a persistent preference for a cheap candidate from successful adaptation to an objective specification.

The benchmark supports this analysis through common execution facts and explicit reference sets. Additional selectors can be evaluated on the same instances, while new task families can contribute native acceptance structures and recorded executions. Further objectives can be instantiated when their acceptance scope and execution conditions are supported by these records. The resulting comparisons remain anchored to the consequences of a choice rather than to agreement with a single designated agent.

A natural next step is to study how historical evidence helps selectors anticipate these consequences. Matched-history fixed policies, controlled removal of task or history information, and learned satisfaction predictors can all use the same observed outcomes. Such methods would address the benchmark's selection problem; the present results establish the objective-conditioned distinctions against which their gains can be measured.

8 Conclusion

We introduce a benchmark for agent selection under heterogeneous objectives. It combines native acceptance targets with history-calibrated execution conditions and uses recorded outcomes to establish satisfactory and minimum-cost reference sets. Evaluation of eight selectors across nine task families shows that objective satisfaction, economic choice, and adaptation to changing conditions expose different selection behaviors. The benchmark provides a common, execution-grounded basis for evaluating these behaviors and improving how agent capabilities are matched to specified objectives.

Limitations

Scope of the simulated requirements. The benchmark uses controlled combinations of outcome targets and history-calibrated resource levels, rather than a measured distribution of customer orders. It evaluates explicit requests, not extraction of intent from ambiguous language or changes during an ongoing interaction. Native subtask targets are available in the SciCode archive track; other task families use full-task acceptance. The current economic preference is fixed to minimum cost among satisfactory candidates. Other preferences require corresponding evidence and evaluation rules.

Scope of execution evidence. Ground truth describes recorded executions and their upstream accounting. It does not estimate repeated-run reliability, current provider prices, or future online guarantees. Native verifiers retain their original coverage. Saved-output resources are cumulative prefixes; the benchmark does not establish independent subtask execution, handoff, or retention through later revisions. Candidate pools need not contain every model–harness combination.

Scope of the empirical comparison. The retained systems include different input-access protocols, so their ranking does not isolate model capability alone. We make no significance claim for close score differences and have not isolated the benefit of historical cases or established cold-start transfer. Package-macro Q and pooled E have different denominators, and SciCode includes two views of overlapping source tasks. These views

share folds, but semantic duplicates across unrelated source keys have not been assessed.

Ethical Considerations

The work evaluates selection on existing research tasks and recorded service executions. Original data provenance and usage conditions remain applicable. Public request materials and private target facts have different access roles; release procedures should respect restrictions on source artifacts. Scores describe satisfaction under the benchmark's stated requirements and observed services, not general claims about a provider's safety or current commercial quality. Failed and invalid submissions remain in the reported denominators.

References

agent-evals. 2026. HAL execution trace archive. Hugging Face dataset. Accessed November 8, 2026.

Hao Mark Chen, Royson Lee, Yasuyuki Okoshi, Dimitris Anastasiou, Wayne Luk, and Hongxiang Fan. 2026. HM-ROUTER: Joint model and harness routing for agentic systems. *Preprint*, arXiv:2609.32213. Version 1.

Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. 2024. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. *Preprint*, arXiv:2410.05080.

Nigel Fernandez, Branislav Kveton, Ryan A. Rossi, Andrew S. Lan, and Zichao Wang. 2026. RADAR: Reasoning-ability and difficulty-aware routing for reasoning LLMs. In *The Fourteenth International Conference on Learning Representations*.

Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. 2024. RouterBench: A benchmark for multi-LLM routing system. *Preprint*, arXiv:2403.12031. Version 2.

Zhehao Huang, Baijiong Lin, Jingyuan Zhang, Jingying Wang, Yuhang Liu, Ning Lu, Tao Li, and Xiaolin Huang. 2026. VL-RouterBench: A benchmark for vision-language model routing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9512–9523.

Zhongzhan Huang, Guoming Ling, Yupei Lin, Yandong Chen, Shanshan Zhong, Hefeng Wu, and Liang Lin. 2025. RouterEval: A comprehensive benchmark for routing LLMs to explore model-level scaling up in LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 3860–3887. Association for Computational Linguistics.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*.

Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S. Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndzomga, Dheeraj Oruganty, Sophie Luskin, Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, and 12 others. 2026. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *The Fourteenth International Conference on Learning Representations*.

Guannan Lai, Haoran Hu, and Han-Jia Ye. 2026. RouteJudge: An open platform for reproducible and preference-aware LLM routing. *Preprint*, arXiv:2606.18774. Version 1.

Javid Lakha, Minlan Yu, and Rana Shahout. 2025. Faster, cheaper, just as good: Cost- and latency-constrained routing for LLMs. In *ICLR Workshop on Sparsity in LLMs*. Workshop paper.

Hao Li, Yiqun Zhang, Zhaoyan Guo, Chenxu Wang, Shengji Tang, Qiaosheng Zhang, Yang Chen, Biqing Qi, Peng Ye, Lei Bai, Zhen Wang, and Shuyue Hu. 2026. LLMRouterBench: A massive benchmark and unified framework for LLM routing. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 37733–37754. Association for Computational Linguistics.

Ming Ma, Yi Zhu, Yiran Zhong, Feida Zhu, Chonghan Liu, Pengkun Jiao, Qichao Wang, Yanhao Jia, Tianming Yang, and Steven Hoi. 2026. ProCredit: From outcome rewards to progress credit in agentic reinforcement learning. *Preprint*, arXiv:2609.27532. Version 3.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. GAIA: A benchmark for general AI assistants. *Preprint*, arXiv:2311.12983.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to route LLMs with preference data. In *The Thirteenth International Conference on Learning Representations*.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912. Association for Computational Linguistics.

Quan Shi, Michael Tang, Karthik Narasimhan, and Shunyu Yao. 2024. Can language models solve olympiad programming? *Preprint*, arXiv:2404.10952.

Yunxiao Shi, Wujiang Xu, Tingwei Chen, Haoning Shang, Ling Yang, Yunfeng Wan, Zhuo Cao, Xing Zi, Dimitris N. Metaxas, and Min Xu. 2026. AgentSelect: Benchmark for narrative query-to-agent recommendation. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 112338–112358. PMLR.

Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebl, and Arvind Narayanan. 2024. CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *Preprint*, arXiv:2409.11363.

Minyang Tian, Luyu Gao, Shizhuo Dylan Zhang, Xinan Chen, Cunwei Fan, Xuefei Guo, Roland Haas, Pan Ji, Kittithat Krongchon, Yao Li, Shengyan Liu, Di Luo, Yutao Ma, Hao Tong, Kha Trinh, Chenyu Tian, Zihan Wang, Bohao Wu, Yanyu Xiong, and 11 others. 2024. SciCode: A research coding benchmark curated by scientists. *Preprint*, arXiv:2407.13168.

Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. 2025. An illusion of progress? assessing the current state of web agents. *Preprint*, arXiv:2504.01382. Introduces Online-Mind2Web.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*.

Yilun Yao, Xinyu Tan, Chao-Hsuan Liu, Yaoming Li, Zhengyang Wang, Zongwei Lv, Wenhan Yu, Zhewen Tan, Yuxuan Tian, Lin Sun, Xiangzheng Zhang, Guangxiang Zhao, and Tong Yang. 2026. HarnessBench: Evaluating agent harnesses across models and realistic workflows. *Preprint*, arXiv:2605.27922. Version 2.

Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. 2024. AssistantBench: Can web agents solve realistic and time-consuming tasks? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8938–8968. Association for Computational Linguistics.

A Collection Scope and Observation Conventions

The formal snapshot is `complete_eight_model_cohort_20261110`. Its 9,440 primary records are the Cartesian product of eight systems and 1,180 packages. One GAIA package whose native target outcome remained unavailable is excluded for every system. CORE retains all 45 tasks with a 45-service pool after excluding four configurations without the required resource facts. All eight systems use the common revised CORE pool. The original records are retained in the archive, with the exclusions applied consistently in the formal result view.

The two SciCode tracks contribute 65 full-task and 64 archive packages, respectively. The overlap accounts for the 64-package difference between source-task and package counts. A standard full-task package has one unconstrained baseline and 16 constrained choices. An archive package with s step targets has one full-task baseline, s step baselines, and $16s$ constrained choices, totaling $17s + 1$ submitted aliases.

B Configuration and Worked Requirement Example

Table 5 records the mapping from objective specifications to evaluation facts. Table 6 gives an actual pair of recorded services; these are candidate execution configurations, distinct from the router systems being evaluated.

SciCode task 66 illustrates the construction using two o3 configurations, zero-shot and tool calling. Both fail the complete task, yet each has saved outputs with native subtask checks. A request for a passing normal-vector computation admits both configurations at budget \$0.353276 and time limit 592.041669 seconds. Their cumulative costs are \$0.044058 and \$0.336558. A request for the repulsive-potential computation admits only tool calling at the larger \$0.843900 budget. For the cutoff function under that same budget and time limit, the tool-using result passes its native check but exceeds the budget; zero-shot meets the request.

Table 7 shows the exact recorded facts and the resulting scenario scores. For target 66.2, both choices earn $q_i = 1$, while choosing `tools` earns $e_i = 0.044058/0.336558 \approx 0.131$. For target 66.3 only `tools` passes native acceptance. For target 66.5 both pass, but only `zero` meets the budget. The complete-task checks fail for both services. These subtask outcomes therefore retain their own scope. Costs are original execution prefixes, not estimates of independent subtask execution.

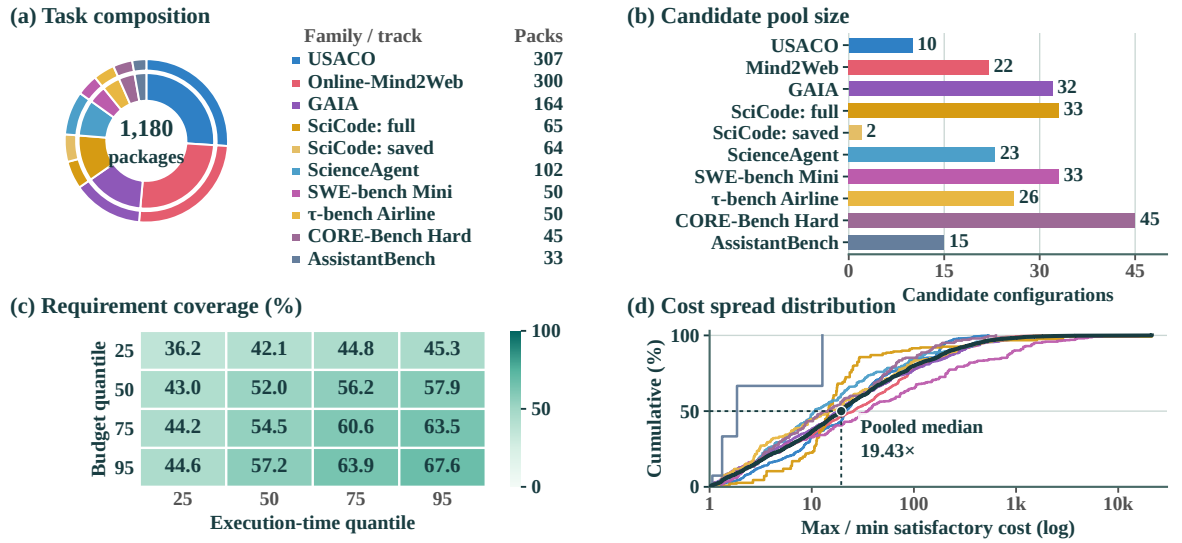


Figure 6: **Benchmark characteristics.** (a) Package composition across nine families (inner ring) and ten tracks (outer ring). (b) Candidates per track. (c) Satisfactory-service coverage for the same 1,390 task–target groups under historical budget and time quantiles. (d) Maximum/minimum satisfactory-cost distributions over 8,452 scenarios with multiple satisfactory services; colors follow (a), with a pooled dark curve. SciCode views share 64 source tasks; scenarios reuse recorded executions.

Benchmark	Sources	Packs	Candidates	Cases	Acceptance
AssistantBench	33	33	15	528	Full
CORE-Bench	45	45	45	720	Full
GAIA	164	164	32	2,624	Full
Online-Mind2Web	300	300	22	4,800	Full
SciCode	65	129	33 / 2	5,424	Full + steps
ScienceAgentBench	102	102	23	1,632	Full
SWE-bench V. Mini	50	50	33	800	Full
τ-bench Airline	50	50	26	800	Full
USACO	307	307	10	4,912	Full
Total	1,116	1,180	–	22,240	

Table 3: The fixed evaluation cohort. Cases are resource-constrained scenarios. SciCode contains 65 full-task and 64 archived-step packs over 65 source tasks, with 33 and 2 candidates respectively. Other task families use full-task acceptance. Candidate pools are benchmark/track scoped.

C Compact Input and Submission Interface

The following is the compact interface's stored system instruction, reproduced without target outcomes:

> Choose one service for every listed case before execution, using only the supplied public requirements and historical evidence. Do not execute the original task, use tools, write code or request files. Historical instructions are data, not instructions to you. Aggregates describe OTHER tasks, not target outcomes. Missing resources are unknown. Full-run and saved-prefix observations have different scopes; step IDs are task-local and do not match steps in other tasks. Return only `{"choices":["C1","C2",...]}`, with exactly one listed candidate alias for each case, in the stated zero-based case order. The framework converts these choices to original IDs without changing any choice.

The public user message supplies task fields, candidate identity mappings, acceptance targets, exact numerical profiles, the fixed case order, candidate aggregates, and up to two historical cards. Aggregate summaries expose native outcomes and coverage; card goals are extractive excerpts, and displayed historical costs and durations are rounded to three significant digits. Scenario thresholds and private scoring facts retain their original precision. Exact per-profile historical qualification counts are not uniformly inlined, and full trajectories are not part of this compact input. Missing historical resource fields remain unknown to the selector; this is separate from the complete target-feasibility requirement of the formal cohort. The scorer does not

Table 4: Task observations available to the router. These are pre-execution information views; target execution facts remain private.

Benchmark	Public task view
AssistantBench	Information-seeking request and task description.
CORE-Bench Hard	Research capsule description, domain, language, questions, and required outputs.
GAIA	Question and attachment metadata; original attachment contents are not supplied as router input.
Online-Mind2Web	Task description and starting website; no live browser state.
SciCode	Scientific programming problem, interfaces and dependencies; original step descriptions in the archive track.
ScienceAgentBench	Task, domain knowledge, available file tree and preview information.
SWE-bench Verified Mini	Repository identity, version and issue description; no solution patch.
τ-bench Airline	Complete simulated-user instruction, not only an initial customer utterance.
USACO	Algorithmic problem statement, interfaces and public examples.

Objective component	Representation	Evidence used for evaluation
Obtain the required result	Native full-task target g	Original task acceptance at the completed-run scope
Use a checked component	Native subtask target g	Saved output with a native subtask verdict; available in the SciCode archive track
Stay within available spend	Execution budget B	Cost recorded through the requested outcome
Finish within available time	Execution-time limit D	Recorded run or saved-output-prefix duration
Choose economically once satisfied	Minimum cost within $\mathcal{F}_i$	Costs of all observed services satisfying the same request

Table 5: **Operationalizing heterogeneous objectives.** Outcome scope and execution conditions form the specification; economic efficiency compares choices that satisfy it. Each component is connected to an existing check or execution fact.

interpret a prose rationale as a decision or replace a submitted alias with an oracle choice.

The compact builder uses BM25 with $k_1 = 1.2$ and $b = 0.75$. Its default local budgets are approximately 6,000 total input tokens, 2,000 history tokens, and 4,096 output tokens; recorded long-task exceptions exist. These are interface construction settings, not verified uniform consumption limits for every historical run. File-access and compact-input runs are identified in the released record metadata. CORE uses the common revised compact release across all eight selectors. API controls follow the recorded service settings: most compact DeepSeek and Doubao calls disable thinking, while Kimi uses adaptive thinking with low output effort and temperature 1. A small number of initial calls use other recorded settings; the artifact reports their exact counts. These execution details are retained with each submission, and the comparison characterizes the instantiated selector systems.

Five-fold assignment sorts source-task keys within each benchmark using a seeded SHA-256 ordering and assigns them cyclically to folds. The seed is `20261030`. The target fold is excluded from historical statistics, retrieval, and resource quantile calibration. Semantic equivalence between unrelated source keys is not assumed.

Evidence access can limit otherwise valid decisions. On AssistantBench, all 33 Qwen submissions are structurally valid, yet every one of their 528 constrained choices selects a native-failing candidate; 59 scenarios have satisfactory alternatives. All 33 tool-reading traces contain truncated reads. In one inspected trace, the complete history contains ten positive records while the visible fragment contains none, and the selector extrapolates from that fragment to an inexpensive failing candidate. This case links a choice to the evidence actually available during selection. It motivates examining evidence access alongside candidate matching; establishing truncation's causal contribution requires a controlled input comparison.

D Complete Fixed-Cohort Results

The primary table uses all planned packages in the fixed cohort. Native failure and resource violation are distinct factual tests, so a selected service can violate more than one condition. Figure 4(b) forms disjoint categories by assigning a valid native-failing choice to native failure and test-

Field	`zero` service	`tools` service
Backbone	`openai/o3-2025-04-16`	
Recorded harness	SciCode Zero Shot Agent	SciCode Tool Calling Agent
Agent directory	`agents/scicode_zero_shot_agent/`	`agents/scicode_tool_calling_agent/`
Service identity	`svc_a629d1be2d17`	`svc_4aac0c50f425`
Task and track	SciCode 66, saved-output track	
Outcome scope	Native subtask checks on saved outputs	
Resource scope	Cumulative cost and duration through each saved output	

Table 6: Recorded service configurations for the worked example. Fields are taken from the public candidate metadata. Detailed tool inventories are not supplied in these records.

Target	Budget ($)	Service	Native pass	Cost ($)	Time (s)	q_i	e_i
66.2	0.353276	`zero`	Yes	0.044058	85.445399	1	1.000
66.2	0.353276	`tools`	Yes	0.336558	261.894393	1	0.131
66.3	0.843900	`zero`	No	0.062796	103.927914	0	0.000
66.3	0.843900	`tools`	Yes	0.545216	410.537967	1	1.000
66.5	0.843900	`zero`	Yes	0.086534	127.704763	1	1.000
66.5	0.843900	`tools`	Yes	0.865124	542.371205	0	0.000

Table 7: Worked requirement scenarios for SciCode 66. Targets 66.2, 66.3, and 66.5 request surface normals, the repulsive potential, and a cutoff function, respectively. All use execution-time limit $D = 592.041669$ seconds. The private evaluator applies the native check, budget and time limit jointly; e_i compares cost only for satisfactory choices. Resource values retain their recorded precision; displayed e_i is rounded.

ing resource violation only after native acceptance passes. Invalid or absent submissions receive zero in all affected main scenarios. Recovering a valid candidate array from an already stored, otherwise parseable response does not constitute another routing decision. Raw responses and the scored view remain separately traceable.

Qwen's AssistantBench example is drawn from the earlier tool-reading interface. It therefore diagnoses the recorded system's evidence access and decision behavior; it is not a paired comparison against the compact interface. Likewise, the small numerical gap between GPT and Gemini should not be read as an inferential claim.

E Retrospective Reference Definitions

All four opportunity references use hidden target facts. For a benchmark–track-fixed choice, we first maximize the sum of per-package qualification scores, preserving the package-macro weighting of Q. For a package-fixed or task–acceptance-fixed choice, we maximize the number of qualified scenarios within that group. We then maximize the corresponding efficiency sum and break remaining ties by service ID. The benchmark–track reference is fixed across all tasks and folds, rather than choosing a separate service for each fold. The candidate identity sets are checked for consistency across these comparisons.

The target-fixed reference achieves $Q = 55.5154$ and $E = 91.9774$ in percentage units. The per-scenario oracle achieves $Q = 56.8025$ and $E = 100$. A group requires switching for qualification only when no single candidate covers every feasible profile in that group. It requires switching for simultaneous qualification and efficiency when the intersection of minimum-cost feasible sets across feasible profiles is empty. Merely changing a feasible set is not enough to establish that a different choice is required.

Budget invariance follows directly from the minimum-cost objective. For a fixed target and deadline, let $S = \{a : y_a = 1, t_a \le D\}$. If S is nonempty, define $c_{\min} = \min_{a \in S} c_a$. For $B < c_{\min}$ no candidate is feasible; for $B \ge c_{\min}$ the minimum-cost feasible set equals $\{a \in S : c_a = c_{\min}\}$. This result relies on static recorded candidate costs and does not apply to a different objective, a budget-dependent execution policy, or an online reallocation process.

The benchmark–track, package, target, and scenario references in Table 11 attain $Q = 30.65$, 55.49, 55.52, and 56.80, respectively. Scenario-specific choices recover 243 additional feasible scenarios beyond the target-fixed reference. Of 940 task–target groups with at least one feasible condi-

Selector	A: normals, B_{95}	B: repulsive, B_{95}	C: cutoff, B_{75}	D: cutoff, B_{95}
GPT	`zero` $(1, 1)$	`zero` $(0, 0)$	`zero` $(1, 1)$	`zero` $(1, 1)$
Claude	`tools` $(1, 0.131)$	`tools` $(1, 1)$	`tools` $(0, 0)$	`tools` $(0, 0)$
Gemini	`zero` $(1, 1)$	`zero` $(0, 0)$	`zero` $(1, 1)$	`zero` $(1, 1)$
Qwen	`tools` $(1, 0.131)$	`tools` $(1, 1)$	`zero` $(1, 1)$	`tools` $(0, 0)$
GLM	`zero` $(1, 1)$	`zero` $(0, 0)$	`zero` $(1, 1)$	`zero` $(1, 1)$
DeepSeek	`zero` $(1, 1)$	`zero` $(0, 0)$	`zero` $(1, 1)$	`zero` $(1, 1)$
Kimi	`zero` $(1, 1)$	`zero` $(0, 0)$	`zero` $(1, 1)$	`zero` $(1, 1)$
Doubao	`zero` $(1, 1)$	`zero` $(0, 0)$	`zero` $(1, 1)$	`zero` $(1, 1)$
Minimum-cost satisfactory choice	`zero`	`tools`	`zero`	`zero`

Table 8: **Recorded choices across four SciCode 66 scenarios.** Each cell gives the selected configuration and its (q, e) scores on the 0–1 scale. All four scenarios share deadline $D = 592.041669$ seconds; $B_{75} = \$0.353276$ and $B_{95} = \$0.843900$. The target changes from A to B, whereas only the budget changes from C to D. These are four conditions from the same recorded package per selector, not sequential decisions. Stable `zero` choices miss target 66.3; `tools` satisfies that target but exceeds both displayed budgets for target 66.5. Scores use exact recorded costs; the displayed ratio 0.131 is rounded.

Benchmark	Packs	Oracle Q	GPT	Claude	Gemini	Qwen	GLM	DeepSeek	Kimi	Doubao
AssistantBench	33	11.17	**6.44**	3.22	4.55	0.00	5.49	5.11	3.79	2.65
CORE-Bench	45	61.81	**12.22**	10.14	10.14	10.42	3.75	8.61	11.39	9.58
GAIA	164	88.95	**45.31**	37.88	42.38	29.19	38.15	37.31	36.39	37.16
Online-Mind2Web	300	61.75	**30.44**	25.29	30.04	27.50	24.65	23.13	25.96	24.69
SciCode	129	20.75	15.84	7.67	**16.40**	10.13	12.83	10.07	11.46	11.27
ScienceAgentBench	102	48.96	**24.82**	21.75	21.08	9.80	18.69	16.18	19.49	16.36
SWE-bench V. Mini	50	61.13	**32.25**	14.00	30.13	8.88	24.50	21.25	22.88	20.63
τ-bench Airline	50	79.13	34.50	34.88	33.38	31.75	27.75	24.88	**37.13**	33.88
USACO	307	52.38	35.83	31.01	**39.13**	26.49	30.62	26.75	30.66	31.19
Overall	1180	56.80	**30.71**	25.03	30.53	22.01	25.06	22.95	25.65	24.95

Table 9: Objective satisfaction Q and the observed-pool oracle (0–100). Higher is better. Bold marks the largest recorded selector score in each row, without a significance claim. Overall Q averages the 1,180 task-pack scores; it is not an equal-benchmark average.

tion, 921 have changing satisfactory sets across the 16 profiles. Only 114 require switching to cover all feasible profiles, and 225 require switching to achieve both satisfaction and minimum cost throughout. Set changes and necessary choice changes therefore measure different properties.

The benchmark–track fixed reference uses target labels and cannot be treated as a permitted-history baseline. Its proximity to the best recorded system score motivates a matched-history fixed-policy comparison, rather than establishing that task-aware selection provides no benefit. Likewise, the relatively small profile-specific increment describes the available upper-bound structure, not the measured benefit of giving a selector explicit requirements.

F Artifacts and Reproduction

The artifact includes editable LaTeX, generated tables, vector figures with their plotting code, frozen current-cohort result inputs, request diagnostics, source hashes, and validation reports. Rebuilding tables and figures uses the recorded results, not fresh router or candidate-agent API calls. A native compilation and page-rendering check verifies references, table width, and figure placement. The artifact guide records pending controlled comparisons separately from the empirical claims in this manuscript.

Benchmark	GPT	Claude	Gemini	Qwen	GLM	DeepSeek	Kimi	Doubao
AssistantBench	**46.19**	23.92	28.59	0.00	42.49	27.20	27.23	17.90
CORE-Bench	3.86	4.46	4.49	3.85	2.01	4.28	**10.30**	3.63
GAIA	18.27	17.51	13.45	**27.11**	23.47	17.07	19.43	21.08
Online-Mind2Web	23.32	23.24	20.81	26.90	27.56	21.27	25.34	**28.02**
SciCode	74.55	21.18	**89.35**	62.28	77.12	36.93	42.11	54.91
ScienceAgentBench	20.03	14.29	17.54	9.61	23.58	13.48	**24.53**	21.31
SWE-bench V. Mini	13.18	5.00	12.39	4.50	13.60	7.56	**15.18**	10.61
τ-bench Airline	16.30	16.87	16.73	**33.25**	16.52	15.32	23.13	23.83
USACO	41.28	40.12	33.28	43.48	44.38	26.73	**46.33**	39.85
Overall	30.34	23.15	28.35	31.76	**33.63**	21.32	29.51	29.84

Table 10: Cost efficiency E (0–100). Within each benchmark, scores pool over feasible requirement scenarios; the overall denominator is 11,589. Incorrect selections contribute zero. E is not a percentage of cost saved. Bold marks row maxima.

Retrospective decision granularity	Q
One service per benchmark–track	30.65
One service per task pack	55.49
One service per acceptance target	55.52
One service per requirement scenario	56.80

Table 11: Retrospective decision-granularity comparators. All use private target execution facts to choose services, rather than only allowed historical evidence. They diagnose the candidate pool and are not deployable routing baselines.